

PeopleSoft Classic → Fluid Conversion Guide

Developer Reference | Production Edition

PeopleTools 8.54 – 8.61+ | 2026

Core Principle: Reuse Data + Reuse Logic + Rebuild UI

Classic-to-Fluid is not an automatic conversion. There is no Oracle-delivered utility that converts arbitrary custom components. The proven approach is:

- Reuse: Existing Records, Search Records, Component structure
- Reuse: Existing PeopleCode business logic (FieldChange, SaveEdit, RowInit, Workflow)
- Reuse: Existing Security model and Permission Lists
- Rebuild: The UI layer from scratch as a new Fluid page set

Before You Start — Confirm Your Environment

PeopleTools Version Requirements

Fluid capabilities expand significantly across PeopleTools releases. Confirm your version before scoping work:

PeopleTools Version	Fluid Status	Recommendation
8.54	Initial Fluid support	Minimum viable — very limited options
8.55 – 8.57	Stable	Solid baseline for most projects
8.58 – 8.60+	Recommended	Full feature set, preferred for new projects
8.61+	Best modern UX	Latest framework — best responsive & mobile options

How to Check

Navigate to: PeopleTools > Utilities > Administration > PeopleTools Options

Phase 1 Assessment & Planning

Understand the component fully before writing a single line of code

1.1 Should This Component Be Converted?

Not every Classic component is a good Fluid candidate. Use this matrix to prioritise:

Good Candidates — High ROI

Component Type	Convert to Fluid?
Self-service pages (Employee / Manager)	✓ Yes — highest value
Manager approval workflows	✓ Yes
Inquiry / read-only pages	✓ Yes — straightforward
Light maintenance pages	✓ Yes

Harder Candidates — Evaluate First

Component Type	Recommended Strategy
Heavy data-entry grid pages	Evaluate — simplify grid design first
Deep multi-scroll pages (3+ levels)	Redesign — flatten scroll hierarchy
Highly customised JS pages	Rebuild — Classic JS APIs incompatible
Power-user admin / back-office pages	Optional — deprioritise

1.2 Discovery & Assessment Checklist

Document every item below before opening Application Designer:

Functional Review

- Search / Add / Update / Correction modes in use
- Security roles and user audience
- Mobile requirement (phone / tablet / desktop only)
- Integration with workflow or approval framework
- Related Components or Component Interfaces (CIs)

Technical Review

- Number of pages and subpages (list each)
- Scroll levels per page — Fluid works best at 2 or fewer
- Grid count and row volume per grid
- Secondary / popup pages (DoModal calls)
- Derived / work records in use
- FieldChange, SavePreChange, RowInit PeopleCode volume and complexity
- HTML Areas — does each contain custom JavaScript?
- Custom JavaScript: identify Classic-specific API calls
- Any %Response.Write or %Request.GetParameter usage
- Existing Permission Lists with component access

Phase 2 Build the Fluid Component

App Designer: step-by-step page and component construction

⚠ Critical: Do NOT Simply Clone and Change Page Type

Changing an existing Classic page type to Fluid does not produce a usable result. Complex Classic layouts break due to absolute positioning, scroll area differences, and style incompatibilities. The correct approach: create NEW Fluid pages and reuse the same records and fields on them.

2.1 Step 1 — Clone the Component Definition

Keep the original Classic component intact and running throughout the project:

1. Open Application Designer → File > Open > Component → find your Classic component
2. File > Save As → give it a Fluid naming convention:
 - JOB_DATA → JOB_DATA_FL
 - EMP_SELF → EMP_SELF_FL
 - VENDOR_MAINT → VENDOR_MAINT_FL
3. Keep the original Classic component unchanged — parallel deployment strategy (see Phase 5)

2.2 Step 2 — Create New Fluid Pages

4. In Application Designer: File > New > Page

5. Name using the _FL suffix convention: JOB_DATA_FL1, JOB_DATA_FL2, etc.
6. Page Properties > Use Tab > Page Type = Fluid
7. Page Properties > Fluid Tab > set a Page Title (required in Fluid — Classic has none)

i Fluid Header (PeopleTools 8.55+)

Do NOT manually drop PT_HEADER_SUB or PT_HEADERPAGE onto your pages. The Fluid theme framework (Tangerine / Fluid Default) injects the header automatically. Manually adding header subpages is an outdated PT 8.54 practice and causes duplicate headers on modern PeopleTools releases.

2.3 Step 3 — Build the Fluid Layout

All Fluid pages use a GroupBox-first design. Fields cannot exist outside a GroupBox:

Recommended Page Structure

- Header / Page Title GroupBox
- Search Info / Key Fields GroupBox
- Primary Data Section GroupBox (2-column layout for desktop)
- Secondary Details GroupBox (collapsible for cleaner mobile)
- Action Buttons GroupBox (bottom of page, right-aligned)

GroupBox layout settings (GroupBox Properties > Fluid Tab):

- Layout = 2 Column for main data sections (collapses to 1 on mobile)
- Layout = 1 Column for simple or mobile-first pages
- Enable Collapsible for secondary/optional sections
- Style = ps_box for standard bordered containers

2.4 Step 4 — Add Fields (Reuse Existing Records)

Drag fields from the same records used in Classic. No new records needed for a straight conversion:

```
/* Examples: reusing existing records unchanged */
PERSONAL_DATA.NAME
PERSONAL_DATA.BIRTHDATE
JOB.DEPTID
JOB.LOCATION
JOB.JOBCODE
DERIVED_WORK.MY_BUTTON /* derived/work records also reusable */
```

2.5 Step 5 — Classic → Fluid Control Mapping

Classic Control	Fluid Equivalent	Notes
Grid	Fluid Grid	Enable Fluid Grid mode in Grid Properties
Scroll Area	Fluid Grid or Child Page	Flatten if > 2 scroll levels
Secondary Page (DoModal)	Fluid Modal Page	Requires PeopleCode change — see Phase 3
Toolbar Push Button	Related Actions / Page Button	Apply ps-button-primary style class
HTML Area	Fluid HTML Area	Review embedded JS — Classic APIs differ
Long Edit Box	Text Area (Fluid type)	psc_textarea style class
Dropdown List	Drop Down (Fluid)	No change to underlying record field
Tabs	Tabs or Accordion	Accordion preferred for mobile

2.6 Component Properties — Enable Fluid Mode

8. Open your new _FL component in Application Designer
9. Component Properties > Fluid Tab:
 - Fluid Mode = Yes (☒ required)
 - Small Form Factor = check if phone support needed
 - Navigation Collection = assign for NavBar visibility
10. Component Properties > Internet Tab: Start Page = your first _FL page
11. Pages list: replace all Classic page references with _FL pages
12. Search Record: Option A — reuse existing (fastest); Option B — create new search record for mobile-friendly keyword search

Phase 3 Grids & Search Strategy

The two areas that most often break Classic → Fluid migrations

3.1 Grid Migration Strategy

Heavy grid pages are the most common source of Fluid conversion problems. Default rule: simplify before you migrate.

Grid Design Principles

- Use grids only when tabular comparison of multiple rows is genuinely needed
- Reduce columns — aim for 5–7 visible columns maximum on a Fluid grid
- Enable paging — avoid loading hundreds of rows into a single grid

- Use collapsible row detail — show summary in grid, full detail in expanded row or modal
- Consider summary / detail pattern: grid as summary list, separate Fluid page for row detail
- Heavy grid pages (15+ columns, complex inline editing) often need full redesign

Grid Properties for Fluid

13. Grid Properties > Fluid Tab > Grid Style = Fluid Grid
14. Enable Allow Personalization if users should reorder/hide columns
15. Set Freeze Columns for key identifier columns on wide grids
16. Set Page Size (rows per page) — 10–25 is appropriate for most mobile contexts

✗ What NOT to Do with Grids

Do not carry a 15-column Classic grid unchanged into Fluid and expect it to render well on tablet or phone. Do not rely on horizontal scroll as the primary navigation pattern. Redesign the grid UX as part of the Fluid conversion — this is expected effort, not a defect.

3.2 Search Page Strategy

Option	Approach	Best For
A	Reuse existing Classic Search Record	Simple conversions — fastest path
B	Create new Search Record with keyword fields	Mobile-first, improved search UX

Enable Keyword Search on the component for a Fluid-style search experience (Component Properties > Search tab > Enable Keyword Search).

Phase 4 PeopleCode Adaptation

What to reuse, what to review, what to rewrite

4.1 What Can Be Reused Without Changes

The majority of business logic carries forward unchanged:

- FieldChange — works identically in Fluid
- SaveEdit — no changes needed
- SavePreChange / SavePostChange — no changes needed
- RowInit — works in Fluid; review display-only logic
- Workflow / Approval Framework PeopleCode — unchanged
- Component Interface calls — unchanged

- Application Engine / Integration Broker — unaffected

4.2 Logic That Often Needs Review or Rewrite

Logic Type	Risk	Why / Action Required
Show/Hide based on pixel layout	High	Fluid uses responsive layout — pixel logic invalid
JavaScript DOM manipulation	High	Different rendering engine — rewrite required
DoModal() popup calls	High	Must replace with Fluid modal pattern
Grid row display hacks	Medium	Fluid grid renders differently — review each
RowInit with Classic display assumptions	Medium	Review for hardcoded field positions
%Response.Write / %Request.GetParameter	High	Bypasses Fluid engine — refactor to HTML Area

4.3 Modal / Secondary Page Pattern

Replace Classic DoModal() calls with the Fluid modal framework:

```

/* — Classic pattern (remove) ————— */
DoModal(Page.MY_SECONDARY_PAGE, "Title", -1, -1);

/* — Fluid replacement ————— */
import PT_RCF_MODAL_CONFIG:ModalConfig;

Local PT_RCF_MODAL_CONFIG:ModalConfig &oConfig;
&oConfig = create PT_RCF_MODAL_CONFIG:ModalConfig();
&oConfig.Title      = "My Modal Title";
&oConfig.TargetPage = Page.MY_SECONDARY_FL;
&oConfig.TargetPanel = Component.MY_COMPONENT_FL;
DoModalX(&oConfig);

```

4.4 JavaScript Migration

Classic JS Pattern	Action	Fluid Replacement
addKeyListener()	Replace	Fluid keyboard event handlers
submitAction_win0()	Replace	pAction_win0() Fluid equivalent
setDefault()	Remove	Use FieldChange PeopleCode instead
document.getElementById()	Review	Use PT Fluid JS API selectors from PT_COMMON.JS

General DOM manipulation

Review

PT_COMMON.JS Fluid API — validate per function

Phase 5 Security & Navigation

Permission Lists, Content References, Tiles, Navigation Collections

5.1 Permission Lists

17. Security > Permissions & Roles > Permission Lists
18. Find each Permission List with access to the Classic component
19. Pages Tab: add the new _FL component with the same access type
20. Validate: Menu access, Component access, Tile visibility, Homepage security

Reuse existing Permission Lists wherever possible — avoid creating duplicate security objects.

5.2 Content Reference

21. Navigate: PeopleTools > Portal > Structure and Content > Portal Objects > Content References
22. Add New Content Reference:
 - Label: user-facing descriptive name
 - Node: LOCAL_NODE
 - URL Type: PeopleSoft Generic URL
 - URL: c/COMPONENT_NAME.COMPONENT_NAME_FL.GBL
 - Valid From / To: set as appropriate
23. Assign to appropriate Navigation Collection or Portal menu folder

5.3 Create a Fluid Tile

24. PeopleTools > Portal > Tiles > Add New Tile
25. Tile Type = Content Reference Tile → link to your new cref
26. Configure tile size: Small (1×1) / Medium (2×1) / Large (2×2)
27. Select icon from PT Icon Library (delivered SVG icons)
28. Optional: Live Tile / Badge showing record counts via query
29. Assign tile to Homepage via: PeopleTools > Portal > Homepage Administration

5.4 Navigation Collections (NavBar)

30. PeopleTools > Portal > Navigation Collections
31. Find or create the Navigation Collection for your functional area
32. Add your Content Reference to the Collection
33. Set display sequence for NavBar ordering

Phase 6 Responsive Design

Mobile-first principles for Fluid page layout

6.1 Mobile Design Rules

Fluid pages must work on phone, tablet, and desktop. Design mobile-first, then enhance for larger screens:

Avoid These Patterns

- ✗ 30+ fields on a single page — split into sections or pages
- ✗ 15-column grids — reduce to 5–7 visible columns maximum
- ✗ Tiny buttons — use large touch targets (minimum 44×44 px effective area)
- ✗ Excessive vertical scrolling — use accordion sections and pagination
- ✗ Horizontal scroll as primary navigation — freeze key columns instead

Prefer These Patterns

- ✓ 1-column layout on mobile (GroupBox Layout = 1 Column, or 2-col that collapses)
- ✓ Clear action buttons with distinct labels
- ✓ Progressive disclosure — show summary first, detail on demand
- ✓ Collapsible GroupBox sections for optional or secondary data
- ✓ Paginated grids — 10–25 rows per page

6.2 Responsive Breakpoints

Device	Width	Layout Behaviour	Key Consideration
Desktop	> 1024 px	Full multi-column layout	Test all grid columns visible
Tablet	768–1024 px	2-col may collapse to 1-col	Test modal / popup sizing
Mobile	< 768 px	Single column, stacked	Touch targets, no hover

i CSS Style Class Note

Delivered style classes (such as psc_hide-xs, psc_hide-lg, ps-button-primary) vary between PeopleTools releases and theme configurations. Always verify class names against the stylesheet shipped with your specific PeopleTools version using browser DevTools on a delivered Fluid page — do not copy class names from documentation written for a different release.

Phase 7 Testing

Functional, UI, security — all viewports and all user roles

7.1 Functional Testing Checklist

- Search — keyword and standard modes
- Add new row / new component instance
- Update existing record
- Save — all paths (normal save, save with workflow trigger)
- Delete row (if applicable)
- Prompt tables / lookup fields
- Workflow launch / approval submit
- Grid: add row, delete row, page forward/back
- Modal / popup open and close with data save
- Back button and browser history behaviour

7.2 UI & Responsive Testing

Test Area	Desktop	Tablet	Mobile
Page layout and field alignment	✓ Required	✓ Required	✓ Required
Grid columns and scroll	✓ Required	✓ Required	✓ Required
Modal / popup sizing	✓ Required	✓ Required	⚠ Review
Tile and homepage display	✓ Required	✓ Required	✓ Required
NavBar / Navigation Collection	✓ Required	✓ Required	✓ Required
Action buttons touch targets	✓ Required	✓ Required	✓ Required

7.3 Security Testing

- Each user role can access the Fluid component via Tile and NavBar
- Field-level security (display-only / hidden fields) correct per role
- Users without access cannot see the Tile on Homepage
- Approval / workflow roles function correctly in Fluid

Common Production Issues & Fixes

Issue	Root Cause	Fix
Page loads in Classic inside Fluid shell	Component not Fluid enabled	Component Properties > Fluid Tab > Fluid Mode = Yes
Fields overlap on page	Poor GroupBox layout	Rebuild sections using proper Fluid GroupBox layout
Slow page performance	Too many grids / large row sets	Add paging, reduce columns, simplify grids
Search page confusing on mobile	Old Classic search record	Create new search record or enable Keyword Search
Buttons misplaced or unstyled	Page layout / missing style class	Reposition in action GroupBox; apply ps-button-primary
Duplicate header on page	PT_HEADER_SUB manually added	Remove subpage — theme handles header automatically
DoModal crash or missing popup	Classic DoModal() not replaced	Replace with PT_RCF_MODAL_CONFIG / DoModalX pattern
JS errors in browser console	Classic JS APIs called	Rewrite using Fluid PT_COMMON.JS API equivalents

Phase 8 Deployment Strategy

Parallel running, pilot, and feedback cycles

8.1 Parallel Deployment — Never Replace Immediately

Run Classic and Fluid side-by-side throughout the project. This is not optional:

Stage	Users	Action
Phase 1: Build	Developer only	Classic remains live for all users
Phase 2: Pilot	Selected pilot group (5–20 users)	Fluid available; Classic still default
Phase 3: Rollout	All users	Fluid becomes default; Classic link kept temporarily

PeopleSoft Classic → Fluid Conversion Guide Developer Reference		Production Edition 2026
Phase 4: Retire	N/A	Classic component access removed after 1–2 cycles

8.2 Naming Standards for New Objects

Object Type	Convention	Example
Component	ORIGINAL_FL	JOB_DATA_FL
Page	ORIGINAL_FL1, FL2...	JOB_DATA_FL1, JOB_DATA_FL2
Tile	ORIGINAL_TILE	JOB_DATA_TILE
Navigation Menu	AREA_FL	EMPLOYEE_FL
New Search Record	ORIGINAL_SRCH_FL	JOB_DATA_SRCH_FL

Real Project Example — JOB_DATA Conversion

Classic JOB_DATA — Before • 5 tabs, all on one component • 3 large grids (10–15 columns each) • Desktop-only design • Multiple DoModal popups • Heavy custom JavaScript	Fluid JOB_DATA_FL — After • Page 1: Employee Summary (key fields only) • Page 2: Job Details (2-column GroupBox layout) • Page 3: Compensation • Page 4: History Grid (paginated, 7 columns) • Fluid modals replacing all DoModal calls • Much better mobile and tablet UX
---	---

Realistic Effort Estimates

All estimates assume a senior PeopleSoft developer with Fluid experience. Source: real-world delivery data.

Component Complexity	Low	Medium	High
Simple inquiry page	1 day	2 days	3 days

PeopleSoft Classic → Fluid Conversion Guide Developer Reference			Production Edition 2026
Light maintenance page	2 days	1 week	2 weeks
Medium multi-page component	1 week	2 weeks	3 weeks
Complex transaction page (grids, modals)	2 weeks	1 month	2 months
Heavily customised JS / multi-scroll	1 month	2 months	3 months

Add 20–30% to estimates for: first Fluid project (learning curve), incomplete Classic documentation, or environments below PeopleTools 8.57.

Production Readiness Checklist

Complete every item before go-live. Use as a sign-off document with your technical lead.

#	Area	Checklist Item
1	Environment	☐ PeopleTools version confirmed — 8.55+ recommended
2	App Designer	☐ All _FL pages have Page Type = Fluid
3	App Designer	☐ Component Fluid Tab: Fluid Mode = Yes
4	App Designer	☐ Pages list updated — no Classic page references remain
5	App Designer	☐ Header subpage NOT manually added (theme delivers header)
6	App Designer	☐ All fields inside GroupBoxes with Fluid layout set
7	App Designer	☐ Grids: Fluid Grid mode enabled; pagination configured
8	PeopleCode	☐ All DoModal() calls replaced with Fluid modal pattern
9	PeopleCode	☐ Classic JS APIs reviewed and replaced with Fluid equivalents
10	PeopleCode	☐ %Response.Write / %Request.GetParameter removed or refactored
11	PeopleCode	☐ RowInit / FieldEdit reviewed for Classic display assumptions
12	Security	☐ Permission Lists updated with new _FL component
13	Security	☐ Tile visibility validated per user role
14	Navigation	☐ Content Reference created and assigned
15	Navigation	☐ Tile created and assigned to correct Homepage
16	Navigation	☐ Navigation Collection updated for NavBar
17	Testing	☐ Functional: Search, Add, Update, Save, Delete tested

18	Testing	☐ Workflow and approval paths tested
19	Testing	☐ Responsive: Desktop, Tablet, Mobile tested
20	Testing	☐ Back button and browser history verified
21	Deployment	☐ Classic component still accessible during pilot phase
22	Deployment	☐ Naming standards followed for all new objects

What NOT To Do

✗ Common Mistakes That Derail Projects

These are the most frequent causes of Classic-to-Fluid project failures observed in production environments:

- ✗ Assume an automatic converter exists — it does not for custom components
- ✗ Copy Classic page pixel-for-pixel into Fluid — redesign the layout
- ✗ Keep giant 15-column Classic grids unchanged in the Fluid page
- ✗ Ignore phone and tablet testing until go-live
- ✗ Skip security validation after migration
- ✗ Replace Classic immediately — always run in parallel first
- ✗ Manually add PT_HEADER_SUB — causes duplicate headers on PT 8.55+
- ✗ Carry Classic JS (addKeyListener, submitAction_win0) into Fluid unchanged

✓ Document Information

This is consolidated Developer Reference for PeopleSoft Classic-to-Fluid Conversion. It reflects real-world delivery practice on PeopleTools 8.54 through 8.61+. Validate steps against your specific PeopleTools release notes before implementation — Oracle updates Fluid framework behaviour between patch releases.